

Lesson 14: Quantum Optimization for Portfolio Selection

Lesson 14: Quantum Optimization for Portfolio Selection Covariance, quadratic programming, SamplingVQE, and result interpretation

Dr. Syed Muzamil Basha

School of Computer Science and Engineering, REVA University

Learning Outcomes

- Explain the role of expected returns, covariance and risk.
- Understand how portfolio selection becomes an optimization problem.
- Describe where a variational quantum routine enters the workflow.
- Interpret the notebook's reported portfolio result carefully.

Why Portfolio Optimization?

- A portfolio balances expected reward against risk.
- Covariance captures how asset returns vary together.
- A budget constraint limits how many assets can be selected.
- The resulting combinatorial optimization problem can be encoded as a quadratic program.

Synthetic Data in the Notebook

- Four tickers are used: AAPL, GOOGL, MSFT and TSLA.
- The RandomDataProvider generates synthetic market data for the selected date range.
- A covariance matrix is computed.
- The notebook uses the mean of covariance columns as `expected_returns`.

Reading the Covariance Matrix

AAPL GOOGL MSFT TSLA

- Diagonal entries represent each asset's own variance-like quantity in the matrix.
- Off-diagonal entries capture pairwise covariance.
- Larger positive covariance means stronger co-movement in the generated data.

Build the Portfolio Optimization Problem

```
num_assets = 4
start = datetime(2022, 1, 1)
end = datetime(2023, 1, 1)
data_provider = RandomDataProvider(
    tickers=['AAPL', 'GOOGL', 'MSFT', 'TSLA'],
    start=start, end=end, seed=42)
data_provider.run()
stock_data = data_provider.get_covariance_matrix()
expected_returns = np.mean(stock_data, axis=0)
risk_factor = 0.5
budget = 2
portfolio = PortfolioOptimization(
    expected_returns=expected_returns,
    covariances=stock_data,
    risk_factor=risk_factor,
    budget=budget)
quadratic_program = portfolio.to_quadratic_program()
ansatz = TwoLocal(rotation_blocks=["ry", "rz"], entanglement_blocks="cz")
optimizer = COBYLA(maxiter=100)
sampler = Sampler()
sampling_vqe = SamplingVQE(sampler=sampler, ansatz=ansatz, optimizer=optimizer)
```

Quadratic Program $\rightarrow$ SamplingVQE $\rightarrow$ MinimumEigenOptimizer

- TwoLocal supplies a parameterized circuit.
- COBYLA is the classical optimizer.
- SamplingVQE searches for a low-energy solution.
- MinimumEigenOptimizer adapts the eigenvalue routine to the optimization problem.

Optimal Portfolio Allocation: []

- The notebook records an empty list of selected assets.
- This is the observed output and must be reported as such.
- It is noteworthy because the optimization problem was created with `budget=2`.
- Therefore, learners should distinguish the intended problem definition from the actual result returned by this notebook execution.

How to Read This Result as a Learner

- The workflow itself is clear even though the recorded final selection is empty.
- Possible causes require debugging/validation of the exact package versions, optimizer behavior, or problem encoding.
- The lesson should not silently replace the recorded result with a guessed portfolio.

Beginner Activity

- Print the quadratic program and identify decision variables.
- Inspect the optimizer result object.
- Verify whether the interpreted solution satisfies the budget constraint.
- Then experiment with the circuit or optimizer settings.

- Portfolio selection can be formulated as a quadratic optimization problem.
- The notebook uses synthetic data and four assets.
- SamplingVQE is used inside MinimumEigenOptimizer.
- The recorded final allocation is the empty list [], despite budget = 2.

Quantum optimization is a hybrid workflow: formulate the problem classically, solve with a variational quantum routine, then interpret the result classically.

Quick Check

- ① What does covariance represent?
- ② What is the budget constraint?
- ③ Why is this a hybrid algorithm?
- ④ What does the recorded empty allocation tell us?